Correcciones para R1 — notebook v3

Campo	Valor
Revisado	`src/migue/practica-002-v3/` (notebook, README, datos y resultados)
Alcance	Secciones 1 a 3
Fecha	10 de noviembre de 2026
Veredicto	Los 10 puntos de la v2 están resueltos. Queda **una decisión que revertir** y dos ajustes

La v3 cierra la retro anterior: `shap` fuera, 20 columnas, filtro de warnings, versiones fijadas y `datos.sha256` verificado. El problema es que la decisión de `categoria` se desvió del plan: el equipo cerró **25 columnas** y tanto el plan (§3.1, §7.3) como el ejemplo de `predecir_venta()` (§7.10) ya están escritos para 25. La retro v2 la ofrecía como opción recomendada, no como cambio de decisión.

P0

1. Volver a `CAT = ["categoria", "product_id"]` (25 columnas)

Por qué. Con 20 columnas el notebook contradice el plan y el ejemplo 7.10, que pasa `categoria` como parte del sample. `predecir_venta()` no falla porque el `reindex` descarta la columna, pero la ignora en silencio: el informe diría que se usa una variable que el modelo nunca vio. Además, la decisión ya está cerrada por el equipo y escrita en el plan.

Qué.

- Celda 3.1: `CAT = ["categoria", "product_id"]` ; sacar `categoria` de `EXCLUIDAS` .
- Celda 3.3: `assert X.shape == (530, 25)` y el Markdown "11 + 9 = 20" pasa a "11 numéricas + 5 dummies de `categoria` + 9 de `product_id` = 25".
- Celda 3.6: **el `assert` actual `not X.T.duplicated().any()` va a fallar**. Debe ser `X.T.duplicated().sum() == 5` , y el Markdown cambia de "la X final no tiene duplicadas" a "las tiene y se conservan por decisión del equipo, con estas consecuencias".
- Celda 2.4: "por eso queda fuera del modelo" → "aporta lo mismo que `product_id` ; se conserva por decisión del equipo".
- `resultados/tabla_variables.csv` : `categoria` pasa a `categórica` , sin motivo de exclusión.
- README: la fila "Variables" y el contrato de `COLUMNAS` pasan de 20 a 25.
- Los dos casos de `predecir_venta()` no cambian: ya incluyen `categoria` .

Cómo redactar 3.6. El diagnóstico debe quedar así, no al revés:

```
Las 5 dummies de `categoria` que entran al modelo son copias exactas de otras tantas de
`product_id` (P0006 a P0010, cada una único producto de su categoría). Se conservan por
decisión del equipo: en el kernel lineal producen colinealidad exacta y en RBF duplican el
peso de la identidad del producto en la distancia. P0002 a P0005 no se duplican porque
`categoria_Alimentos` es la de referencia y sería su suma.
```

P1

2. El README cita 22 páginas y el plan tiene 19

Por qué. El PDF se regeneró; la cifra quedó vieja en la sección de documentación.

Qué. Cambiar "22 páginas" por "19 páginas".

3. El `assert` de versiones es demasiado estricto

Por qué. Exige coincidencia exacta de 11 componentes, incluidos `jupyterlab`, `nbconvert` e `ipykernel`. Cualquier diferencia de parche en las herramientas detiene el notebook en la celda 1, aunque el análisis no cambie. En tu entorno pasó, pero un compañero con `4.6.5` no podría ejecutar nada.

Qué. Limitar la comprobación a las librerías que afectan a los resultados (`numpy`, `pandas`, `scikit-learn`, `joblib`, `matplotlib`, `seaborn`, `mlflow` y Python), o dejar las herramientas como aviso en vez de `assert`.

Cierre

- □ `CAT = ["categoria", "product_id"]`; `X` con (530, 25).
- □ 3.6 ejecuta con 5 duplicadas y las explica.
- □ 2.4 y `tabla_variables.csv` coherentes con 25.
- □ README con 25 columnas y 19 páginas.
- □ 0 errores, 0 warnings y `sha256sum -c` sigue pasando.